

1 Basic

1.1 Default

```

1 #include<bits/stdc++.h>
2 #define int long long int
3 #define all(x) (x).begin(), (x).end()
4 #define INF 1e15+9
5 #define DBG(x) cout<<(#x " = ") <<x<<endl;
6 #define pb push_back
7 #define fastio ios_base::sync_with_stdio(0);cin.
   tie(0);
8 using namespace std;
9 const int maxn = 2e5+5;
10 const int MOD = 1e9+7; // 998244353;
11 typedef pair<int,int> P;
12
13 void solve(){
14     // Do Something...
15 }
16 signed main(){
17     fastio
18     int t=1;
19     cin>>t;
20     while(t--) solve();
21 }

```

2 DataStructure

2.1 Mo

```

1 // Mo's with update
2 // discretization + odd/even optimize
3 #include<bits/stdc++.h>
4 using namespace std;
5 const int maxn = 1e6+6;
6
7 int n,q,bks,sum,L,R;
8 vector<int> v(maxn), cnt(maxn);
9 vector<pair<int,int>> mod;
10 bool cmp(array<int,4> a, array<int,4> b){
11     if(a[0] / bks == b[0] / bks){
12         if(a[1] / bks == b[1] / bks){
13             return a[2] < b[2];
14         }
15         else{
16             return a[1] / bks < b[1] / bks;
17         }
18     }
19     return (a[0] / bks) < (b[0] / bks);
20 }
21 void ADD(int x){
22     if(++cnt[x] == 1) sum++;
23 }
24 void SUB(int x){
25     if(--cnt[x] == 0) sum--;
26 }
27 void MOD(int x){
28     auto [id, val] = mod[x]; // mod_id = x
29     if(id<=R && id>=L){
30         ADD(val);
31         SUB(v[id]);
32     }
33     swap(mod[x].second, v[id]);
34 }
35 int main(){
36     ios_base::sync_with_stdio(0); cin.tie(0);

```

```

37     cin>>n>>q;
38     bks = pow(n, (double)2/(double)3); // bBlock =
   n^(1.5)
39     for(int i=1;i<=n;i++) cin>>v[i];
40     vector<array<int,4>> Q;
41     vector<int> ans(q,-1);
42     int tid = -1, l, r, T = -1;
43     for(int i=0;i<q;i++){
44         char c;
45         cin>>c>>l>>r;
46         if(c == 'Q'){
47             Q.push_back({l, r, tid, i});
48         }
49         else{
50             mod.push_back({l, r}); // replace (id
   =L) with (color=r)
51             tid++;
52         }
53     }
54     L = 1, R = 0;
55     sort(Q.begin(),Q.end(),cmp);
56     for(int i=0;i<Q.size();i++){
57         auto [l,r,ti,ii] = Q[i];
58         while(L > l) ADD(v[--L]);
59         while(R < r) ADD(v[++R]);
60         while(L < l) SUB(v[L++]);
61         while(R > r) SUB(v[R--]);
62         while(T < ti) MOD(++T);
63         while(T > ti) MOD(T--);
64         ans[ii] = sum;
65     }
66     for(int i:ans){
67         if(i!=-1) cout<<i<<"\n";
68     }
69 }

```

2.2 Lazy Seg

```

1 #include<bits/stdc++.h>
2 #define int long long int
3 using namespace std;
4 const int maxn = 2e5+5;
5
6 struct SegTree{
7     int sum[maxn*4], tag[maxn*4];
8     void init(int n){
9         for(int i=0; i<=n*4; i++){
10             sum[i] = 0;
11             tag[i] = 0;
12         }
13     }
14     int L(int x){
15         return (x<<1);
16     }
17     int R(int x){
18         return (x<<1|1);
19     }
20     void pull(int id){
21         sum[id] = sum[L(id)] + sum[R(id)];
22     }
23     void addTag(int id, int l, int r, int k){
24         tag[id] += k;
25         int len = r-l+1;
26         sum[id] += k*len;
27     }
28     void pushTag(int id, int l, int r){
29         int mid = (l+r)>>1;
30         addTag(L(id), l, mid, tag[id]);
31         addTag(R(id), mid+1, r, tag[id]);

```

```

32     tag[id] = 0;
33 }
34 void build(int id, int l, int r, vector<int>
35 &v){
36     if(l == r){
37         sum[id] = v[l];
38         tag[id] = 0;
39         return;
40     }
41     int mid = (l+r)>>1;
42     build(L(id), l, mid, v);
43     build(R(id), mid+1, r, v);
44     pull(id);
45 }
46 void range_update(int id, int l, int r, int
47 ll, int rr, int val){
48     if(ll <= l && r <= rr){
49         addTag(id, l, r, val);
50         return;
51     }
52     pushTag(id, l, r);
53     int mid = (l+r)>>1;
54     if(ll <= mid) range_update(L(id), l, mid,
55 ll, rr, val);
56     if(mid < rr) range_update(R(id), mid+1, r
57 , ll, rr, val);
58     pull(id);
59 }
60 int range_query(int id, int l, int r, int ll,
61 int rr){
62     if(l>=ll && r<=rr) return sum[id];
63     pushTag(id, l, r);
64     int mid = (l+r)>>1;
65     if(ll<=mid) res += range_query(L(id), l,
66 mid, ll, rr);
67     if(mid<rr) res += range_query(R(id), mid
68 +1, r, ll, rr);
69     return res;
70 }
71 }
72 }segtree;
73 // segtree.init(n);
74 // vector<int> v(n+1); (indexed at 1!!!)
75 // segtree.build(1, 1, n, v);
76 // range update + range query

```

2.3 BIT query

```

1 // N = max value, LOGN = Log(N)
2 int bit_search(int v){
3     int sum = 0, pos = 0;
4     for(int i=LOGN; i>=0; i--){
5         if(pos + (1 << i) < N and sum + bit[pos + (1
6 << i)] < v){
7             sum += bit[pos + (1 << i)];
8             pos += (1 << i);
9         }
10    }
11    // may use bit_query to check:
12    // if(que(pos+1) < v) return pos;
13    return pos + 1; // +1 because 'pos' will have
    position of largest value less than 'v'

```

2.4 PBDS

```

1 #include <ext/pb_ds/assoc_container.hpp>
2 #include <ext/pb_ds/tree_policy.hpp>
3 using namespace __gnu_pbds;

```

```

4 template<typename T> using rbt = tree<T,
5     null_type, less<T>, rb_tree_tag,
6     tree_order_statistics_node_update>;
7 int main(){
8     rbt<int> t; //declare
9 }
10 /*
11 不支援重複值 (需要的話可用左推+值來處理)
12 支援set, map之操作
13 find_by_order(k) : 像陣列一樣回傳第 k 個值。(0-
14 based, pointer)
15 order_of_key(k) : 回傳 k 是集合裡第幾大。(0-based
16 )
17 T 資料型別
18 null_type //當作 map 使用的時候要對應什麼資料型
19 態
20 //要當作 set 就用 null_type
21 less<T> // key value 要用什麼方式比較
22 */

```

2.5 Treap

```

1 #include <bits/stdc++.h>
2 #define int long long
3 using namespace std;
4
5 struct node{ // support range reverse, range sum
6     query
7     node *l, *r;
8     int key, val, sum, pri, size, rev;
9     node(int k, int v) : l(0), r(0), key(k), val(
10 v), sum(v), pri(rand()), size(1), rev(0)
11 {}
12 void up();
13 void down();
14 };
15
16 node *merge(node *a, node *b){
17     if(!a || !b) return a ? a : b;
18     if(a->pri < b->pri){
19         a->down();
20         a->r = merge(a->r, b);
21         a->up();
22         return a;
23     }
24     else{
25         b->down();
26         b->l = merge(a, b->l);
27         b->up();
28         return b;
29     }
30 }
31
32 void split(node *o, node *&a, node *&b, int k){
33     // split by key
34     if(!o)
35         a = b = 0;
36     else{
37         o->down();
38         if(o->key < k){
39             a = o;
40             split(o->r, a->r, b, k);
41         }
42         else{
43             b = o;
44             split(o->l, a, b->l, k);
45         }
46     }

```

```

42     o -> up();
43 }
44 }
45 }
46
47 void insert(node *&root, int k, int v){
48     node *a, *b;
49     split(root, a, b, k);
50     root = merge(a, merge(new node(k, v), b));
51 }
52
53 bool erase(node *&o, int k){ // erase T[k]
54     if(!o)
55         return 0;
56     if(o -> key == k){
57         node *t = o;
58         o = merge(o -> l, o -> r);
59         delete t;
60         return 1;
61     }
62     node *t = k < o -> key ? o -> l : o -> r;
63     if(erase(t, k)) return o -> up(), 1;
64     else return 0;
65 }
66
67 void node::up(){
68     size = 1;
69     sum = val;
70     if(l) {
71         //l -> down();
72         size += l -> size;
73         sum += l -> sum;
74     }
75     if(r) {
76         //r -> down();
77         size += r -> size;
78         sum += r -> sum;
79     }
80 }
81
82 void node::down(){
83     if(rev){
84         swap(l, r);
85         if(l) l -> rev ^= 1;
86         if(r) r -> rev ^= 1;
87         rev = 0;
88     }
89 }
90
91 inline int size(node *o){
92     return o ? o -> size : 0;
93 }
94
95 int Rank(node *& root, int val){// Number of
96     // elements smaller than val.
97     node *a, *b;
98     split(root, a, b, val);
99     int res = size(a);
100     root = merge(a, b);
101     return res;
102 }
103
104 void split2(node *o, node *&a, node *&b, int k){
105     // split by size
106     if(!o)
107         a = b = 0;
108     else{
109         o -> down();
110         if(k >= size(o -> l) + 1){
111             a = o;

```

```

110         int nk = k - (size(o -> l) + 1);
111         split2(o -> r, a -> r, b, nk);
112     }
113     else{
114         b = o;
115         split2(o -> l, a, b -> l, k);
116     }
117     o -> up();
118 }
119 }
120
121 node *kth(node *&root, int k){ // find T[k]
122     node *a, *b, *c;
123     split2(root, a, c, k);
124     split2(a, a, b, k - 1);
125     root = merge(a, merge(b, c));
126     return b;
127 }
128
129 void reverse(node *&root, int l, int r){
130     node *a, *b, *c;
131     split2(root, a, b, l - 1);
132     split2(b, b, c, r - l + 1);
133     b -> rev ^= 1;
134     root = merge(a, merge(b, c));
135 }
136
137 int query(node *&root, int l, int r){
138     node *a, *b, *c;
139     split2(root, a, b, l - 1);
140     split2(b, b, c, r - l + 1);
141     b -> down();
142     int res = b -> sum;
143     root = merge(a, merge(b, c));
144     return res;
145 }
146
147 void update(node *&root, int pos, int x){ // let
148     // T[pos] = x
149     erase(root, pos);
150     insert(root, pos, x);
151 }
152
153 signed main(){
154     node *T(nullptr);
155     int n, q;
156     cin >> n >> q;
157     for(int i=1; i<=n; i++){
158         int tmp;
159         cin >> tmp;
160         insert(T, i, tmp);
161     }
162     while(q--){
163         int op;
164         cin >> op;
165         if(op == 1){ // update
166             int pos, x;
167             cin >> pos >> x;
168             update(T, pos, x);
169         }
170         else{ // query
171             int l, r;
172             cin >> l >> r;
173             cout << query(T, l, r) << '\n';
174         }
175     }

```

2.6 DSU set

```

1  /*
2  This DSU supports the following operations :
3      1 x y, Union the sets containing x and y. If
4          they are already in the same set, ignore.
5      2 x y, Move x to the set containing y. If
6          they are already in the same set, ignore.
7      3 x, Return the number of elements and the
8          sum of elements in the set containing x.
9  */
10 #include <bits/stdc++.h>
11 #define int long long
12 using namespace std;
13 const int maxn = 2e5+5;
14
15 int realIndex[maxn];
16 int p[maxn];
17 int ele[maxn];
18 int sum[maxn];
19 int cnt;
20
21 void init(int n){
22     cnt = n + 1;
23     for(int i=1;i<=n;i++){
24         p[i] = i;
25         ele[i] = 1;
26         sum[i] = i;
27         realIndex[i] = i;
28     }
29 }
30
31 int find(int x){
32     if(p[x] == x)
33         return x;
34     else
35         return p[x] = find(p[x]);
36 }
37
38 void merge(int x, int y){
39     x = realIndex[x];
40     y = realIndex[y];
41     int px = find(x), py = find(y);
42     if(px == py)
43         return;
44     p[px] = py;
45     sum[py] += sum[px];
46     ele[py] += ele[px];
47 }
48
49 void moveXToY(int x, int y){
50     if(find(realIndex[x]) == find(realIndex[y]))
51         return;
52     sum[find(realIndex[x])] -= x;
53     ele[find(realIndex[x])] -= 1;
54     realIndex[x] = cnt++;
55     sum[realIndex[x]] = x;
56     ele[realIndex[x]] = 1;
57     p[realIndex[x]] = realIndex[x];
58     merge(x, y);
59 }
60
61 void output(int x){
62     x = realIndex[x];
63     x = find(x);
64     cout << ele[x] << ' ' << sum[x] << endl;
65 }
66
67 signed main(){
68     int n, q;
69     while(cin >> n >> q){
70         init(n);

```

```

68     while(q--){
69         int type;
70         cin >> type;
71         if(type == 1){
72             int x, y;
73             cin >> x >> y;
74             merge(x, y);
75         }
76         else if(type == 2){
77             int x, y;
78             cin >> x >> y;
79             moveXToY(x, y);
80         }
81         else{
82             int x;
83             cin >> x;
84             output(x);
85         }
86     }
87 }
88 }

```

2.7 Per Seg

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 struct Per_seg{
4     int id,nn;
5     vector<int> ln,rn,val,root;
6     Per_seg(int n, vector<int> &arr) : nn(n), id
7         (0){
8         root.resize(1);
9         ln.resize(32*n);
10        rn.resize(32*n);
11        val.resize(32*n);
12        root[0] = build(0, nn-1, arr);
13    }
14    int build(int l, int r, vector<int> &arr){
15        int rt = ++id, mid = (l+r)>>1;
16        if(l==r){
17            val[rt] = arr[l];
18            return rt;
19        }
20        ln[rt] = build(l, mid, arr);
21        rn[rt] = build(mid+1, r, arr);
22        val[rt] = val[ln[rt]] + val[rn[rt]]; //
23        pull
24        return rt;
25    }
26    int update(int pre_id, int l, int r, int pos,
27        int v){
28        int rt = ++id, mid = (l+r)>>1;
29        ln[rt] = ln[pre_id];
30        rn[rt] = rn[pre_id];
31        val[rt] = val[pre_id];
32        if(l==r){
33            val[rt] = v;
34            return rt;
35        }
36        if(pos<=mid) ln[rt] = update(ln[pre_id],
37            l, mid, pos, v);
38        else rn[rt] = update(rn[pre_id], mid+1, r
39            , pos, v);
40        val[rt] = val[ln[rt]] + val[rn[rt]]; //
41        pull
42        return rt;
43    }
44    int query2(int o, int l, int r, int ql, int
45        qr){

```

```

39     if(l>=ql && r<=qr) return val[o];
40     int mid = (l+r)>>1;
41     if(qr<=mid) return query2(ln[o], l, mid,
42         ql, qr);
43     else if(ql>mid) return query2(rn[o], mid
44         +1, r, ql, qr);
45     return query2(ln[o], l, mid, ql, qr) +
46         query2(rn[o], mid+1, r, ql, qr);
47 }
48 // ---for outer---
49 int clone(int ver){
50     int rt = ++id;
51     ln[rt] = ln[root[ver]];
52     rn[rt] = rn[root[ver]];
53     val[rt] = val[root[ver]];
54     root.push_back(rt);
55     return rt;
56 }
57 void point_update(int ver, int pos, int
58     newVal) {
59     int newRoot = update(root[ver], 0, nn-1,
60         pos, newVal);
61     // root.push_back(newRoot); // copy
62     root[ver] = newRoot; // replace
63 }
64 int range_query(int ver, int l, int r){
65     // root.push_back(root[ver]); // copy
66     return query2(root[ver], 0, nn-1, l, r);
67 }
68 };
69 /*
70 0-indexed!!
71 Per_seg seg(n, arr); - build
72 */

```

2.8 DSU f

```

1 struct DSU {
2     vector<int> f, siz;
3
4     DSU(){}
5     DSU(int n){
6         init(n);
7     }
8
9     void init(int n){
10         f.resize(n);
11         iota(f.begin(), f.end(), 0);
12         siz.assign(n, 1);
13     }
14
15     int find(int x){
16         while (x != f[x]) {
17             x = f[x] = f[f[x]];
18         }
19         return x;
20     }
21
22     bool same(int x, int y){
23         return find(x) == find(y);
24     }
25
26     bool merge(int x, int y){
27         x = find(x);
28         y = find(y);
29         if (x == y) {
30             return false;
31         }
32         siz[x] += siz[y];

```

```

33         f[y] = x;
34         return true;
35     }
36
37     int size(int x){
38         return siz[find(x)];
39     }
40 };
41
42 // USE: DSU dsu(n);

```

2.9 SparseTable

```

1 const int MAXN = 5e5 + 5;
2 const int lgN = 20;
3
4 struct SP{ // 0-based
5     vector<int> Sp[lgN];
6     void build(int n, vector<int>& a){
7         for(int i=0;i<n;i++){
8             Sp[0].push_back(a[i]);
9             for(int h=1;h<lgN;h++){
10                 int len = (1 << (h - 1)), i = 0;
11                 for(; i + len < n; i++){
12                     Sp[h].push_back(max(Sp[h-1][i],
13                         Sp[h-1][i+len]));
14                 }
15                 for(; i < n; i++){
16                     Sp[h].push_back(Sp[h-1][i]);
17                 }
18             }
19         }
20         int query(int l, int r){
21             int lg = __lg(r - l + 1);
22             int len = (1 << lg);
23             return max(Sp[lg][l], Sp[lg][r - len +
24                 1]);
25         }
26     }
27 };

```

2.10 01trie

```

1 // for xor
2 const int N = 1e5+5;
3 int tot,trie[41*N][2],n; // need reset tot, trie
4 int find(int x){
5     int p=0,sum=0;
6     for(int i=40;i>=0;i--){
7         int id=(x>>i)&1;
8         if(trie[p][id^1]){ // here, choose id^1
9             sum=sum*2+1;
10            p=trie[p][id^1];
11        }
12        else{
13            sum=sum*2; // here, choose id
14            p=trie[p][id];
15        }
16    }
17    return sum;
18 }
19 // fixed:
20 void insert(int x){
21     int p=0;
22     for(int i=40;i>=0;i--){
23         int id=(x>>i)&1;
24         if(!trie[p][id]) trie[p][id]=++tot;
25         p=trie[p][id];
26     }
27 }

```

2.11 Dy Seg

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Node {
5     unordered_map<int, int> mp; // 記錄頻率
6     int tag = 0;
7     int l, r;
8     Node* left = nullptr;
9     Node* right = nullptr;
10 };
11
12 Node* build(int l, int r) {
13     Node* node = new Node();
14     node->l = l;
15     node->r = r;
16     return node;
17 }
18
19 void pushdown(Node* node) { // push tags
20     if (node->tag) {
21         if (!node->left) node->left = build(node->l, (node->l + node->r) / 2);
22         if (!node->right) node->right = build((node->l + node->r) / 2 + 1, node->r);
23         // Update child
24         node->left->mp.clear();
25         node->right->mp.clear();
26         int mid = (node->l + node->r) / 2;
27         int left_len = mid - node->l + 1;
28         int right_len = node->r - mid;
29         node->left->mp[node->tag] = left_len;
30         node->right->mp[node->tag] = right_len;
31         node->left->tag = node->tag;
32         node->right->tag = node->tag;
33
34         node->tag = 0;
35     }
36 }
37
38 // 將 [l, r] 區間內的數字設為 val
39 void range_update(Node* node, int l, int r, int val) {
40     if (l <= node->l && node->r <= r) {
41         node->mp.clear();
42         node->mp[val] = node->r - node->l + 1;
43         node->tag = val; // tag!
44         return;
45     }
46
47     pushdown(node);
48     int mid = (node->l + node->r) / 2;
49
50     if (l <= mid) {
51         if (!node->left) node->left = build(node->l, mid);
52         range_update(node->left, l, r, val);
53     }
54     if (r > mid) {
55         if (!node->right) node->right = build(mid + 1, node->r);
56         range_update(node->right, l, r, val);
57     }
58
59     // push_up
60     node->mp.clear();
61     if (node->left) {
62         for (auto& [key, val] : node->left->mp) {
63             node->mp[key] += val;
64         }
65     }
66     if (node->right) {
67         for (auto& [key, val] : node->right->mp) {
68             node->mp[key] += val;
69         }
70     }
71 }
72
73 unordered_map<int, int> query(Node* node, int l, int r) {
74     if (l <= node->l && node->r <= r) return node->mp;
75
76     pushdown(node);
77     int mid = (node->l + node->r) / 2;
78     unordered_map<int, int> result;
79
80     if (l <= mid && node->left) {
81         auto left_result = query(node->left, l, r);
82         for (auto& [key, val] : left_result) {
83             result[key] += val;
84         }
85     }
86
87     if (r > mid && node->right) {
88         auto right_result = query(node->right, l, r);
89         for (auto& [key, val] : right_result) {
90             result[key] += val;
91         }
92     }
93
94     return result;
95 }
96
97 void recycle(Node* &node) {
98     if (node == nullptr) return;
99     if (node->left) recycle(node->left);
100    if (node->right) recycle(node->right);
101    delete node;
102    node = nullptr;
103 }
104
105 void delete_range(Node* node, int l, int r) {
106     // delete range [l, r]
107     if (l <= node->l && node->r <= r) {
108         node->mp.clear();
109         node->tag = 0;
110         recycle(node->left);
111         recycle(node->right);
112         return;
113     }
114
115     pushdown(node);
116     int mid = (node->l + node->r) / 2;
117     if (node->left && l <= mid) {
118         delete_range(node->left, l, r);
119     }
120     if (node->right && r > mid) {
121         delete_range(node->right, l, r);
122     }
123
124     // push_up
125     node->mp.clear();
126     if (node->left) {
127         for (auto& [key, val] : node->left->mp) {
128             node->mp[key] += val;
129         }
130     }
131     if (node->right) {
132         for (auto& [key, val] : node->right->mp) {
133             node->mp[key] += val;
134         }
135     }
136 }

```

```

129     if (node->right) {
130         for (auto& [key, val] : node->right->mp)
131             node->mp[key] += val;
132     }
133 }
134 }
135
136
137 int main() {
138     int n, q;
139     cin >> n >> q;
140     Node* root = build(1, n);
141
142     while (q--) {
143         int op, l, r, x;
144         cin >> op;
145         if (op == 1) {
146             cin >> l >> r >> x; // set [l,r] to x
147             range_update(root, l, r, x);
148         } else {
149             cin >> l >> r; // query [l,r] every
150                 number's frequency
151             unordered_map<int, int> freq = query(
152                 root, l, r);
153             for (auto& [key, val] : freq) {
154                 cout<<key<<" "<<val<<endl;
155             }
156             delete_range(root, l, r);
157         }
158     }
159 }

```

3 Geometric

3.1 Rectangle Union Area

```

1 const int maxn = 1e5 + 10;
2 struct rec{
3     int t, b, l, r;
4     //t頂·b底·l左·r右邊界點
5 } r[maxn];
6 int n, cnt[maxn << 2];
7 long long st[maxn << 2], ans = 0;
8 vector<int> x, y;
9 vector<pair<pair<int, int>, pair<int, int>>> v;
10 void modify(int t, int l, int r, int ql, int qr,
11     int v) {
12     if (ql <= l && r <= qr) cnt[t] += v;
13     else {
14         int m = (l + r) >> 1;
15         if (qr <= m) modify(t << 1, l, m, ql, qr,
16             v);
17         else if (ql >= m) modify(t << 1 | 1, m, r,
18             ql, qr, v);
19         else modify(t << 1, l, m, ql, m, v),
20             modify(t << 1 | 1, m, r, m, qr, v);
21     }
22     if (cnt[t]) st[t] = y[r] - y[l];
23     else if (r - l == 1) st[t] = 0;
24     else st[t] = st[t << 1] + st[t << 1 | 1];
25 }
26 int main() {
27     cin >> n;
28     for (int i = 0; i < n; i++) {
29         //輸入個個長方形的上下左右界
30         cin >> r[i].l >> r[i].r >> r[i].b >> r[i].t;
31     }
32 }

```

```

27     if (r[i].l > r[i].r) swap(r[i].l, r[i].r);
28     if (r[i].b > r[i].t) swap(r[i].b, r[i].t);
29     x.push_back(r[i].l);
30     x.push_back(r[i].r);
31     y.push_back(r[i].b);
32     y.push_back(r[i].t);
33 }
34 sort(x.begin(), x.end());
35 sort(y.begin(), y.end());
36 x.erase(unique(x.begin(), x.end()), x.end());
37 y.erase(unique(y.begin(), y.end()), y.end());
38 for (int i = 0; i < n; i++) {
39     r[i].l = lower_bound(x.begin(), x.end(),
40         r[i].l) - x.begin();
41     r[i].r = lower_bound(x.begin(), x.end(),
42         r[i].r) - x.begin();
43     r[i].b = lower_bound(y.begin(), y.end(),
44         r[i].b) - y.begin();
45     r[i].t = lower_bound(y.begin(), y.end(),
46         r[i].t) - y.begin();
47     v.emplace_back(make_pair(r[i].l, 1),
48         make_pair(r[i].b, r[i].t));
49     v.emplace_back(make_pair(r[i].r, -1),
50         make_pair(r[i].b, r[i].t));
51 }
52 sort(v.begin(), v.end(), [](pair<pair<int,
53     int>, pair<int, int>> a, pair<pair<int,
54     int>, pair<int, int>> b){
55     if (a.first.first != b.first.first)
56         return a.first.first < b.first.first;
57     return a.first.second > b.first.second;
58 });
59 for (int i = 0; i < v.size(); i++) {
60     if (i) ans += (x[v[i].first.first] - x[v[
61         i - 1].first.first]) * st[1];
62     modify(1, 0, y.size(), v[i].second.first,
63         v[i].second.second, v[i].first.second
64         );
65 }
66 cout << ans << '\n';
67 return 0;
68 }

```

3.2 TheLeastCoverCircle

```

1 const double eps = 1e-10, pi = acos(-1);
2 struct Circle{
3     pdd o;
4     double r;
5 }c;
6 vector<pdd> p;
7 int R, n, r;
8 pdd operator+(pdd a, pdd b){
9     return {a.F + b.F, a.S + b.S};
10 }
11 pdd operator-(pdd a, pdd b){
12     return {a.F - b.F, a.S - b.S};
13 }
14 pdd operator*(pdd a, double b){
15     return {a.F * b, a.S * b};
16 }
17 pdd operator/(pdd a, double b){
18     return {a.F / b, a.S / b};
19 }
20 double operator*(pdd a, pdd b){
21     return a.F * b.S - a.S * b.F;
22 }

```



```

23 int judge(double a, double b){
24     if (fabs(a-b) < eps) return 0;
25     if (a < b) return -1;
26     return 1;
27 }
28 pdd rotate(pdd a, double b){
29     return {a.F*cos(b)+a.S*sin(b), -a.F*sin(b)+a.
        S*cos(b)};
30 }
31 double lens(pdd a, pdd b){
32     double dx = b.F - a.F, dy = b.S - a.S;
33     return sqrt(dx*dx + dy*dy);
34 }
35 pdd intersection(pdd p, pdd v, pdd q, pdd w){//求
    交點
36     pdd u = p - q;
37     double t = w*u/(v*w);
38     return p + v * t;
39 }
40 pair<pdd, pdd> bisector(pdd a, pdd b){//求中垂線
41     pdd p = (a + b) / 2.0;
42     pdd v = rotate(b - a, pi / 2.0);
43     return {p, v};
44 }
45 Circle circle(pdd a, pdd b, pdd c){ //三點求圓
46     auto n = bisector(a, b), m = bisector(a, c);
47     pdd o = intersection(n.F, n.S, m.F, m.S);
48     double r = lens(o, a);
49     return {o, r};
50 }
51 void solve(){
52     p.clear();
53     cin >> n;
54     p.resize(n);
55     //輸入所有的點
56     for (int i = 1; i <= n; i++){
57         cin >> p[i].F >> p[i].S;
58     }
59     random_shuffle(all(p));
60     c = {p[0], 0};
61     for(int i = 1; i <= n; i++){
62         if (judge(c.r, lens(c.o, p[i])) == -1){
63             c = {p[i], 0};
64             for (int j = 0; j < i; j++){
65                 if (judge(c.r, lens(c.o, p[j]))
                    == -1){
66                     c = {(p[i] + p[j]) / 2.0,
                        lens(p[i], p[j]) / 2.0};
67                     for (int k = 0; k < j; k++){
68                         if (judge(c.r, lens(c.o,
                            p[k])) == -1){
69                             c = circle(p[i], p[j]
                                ], p[k]);
70                         }
71                     }
72                 }
73             }
74         }
75     }
76     //c: {圓心, 半徑}
77     if (n == 1) c = {(p[0] + p[1]) / 2.0, 0};
78     cout << setprecision(9) << fixed;
79     cout << -c.o.F << " " << -c.o.S << endl;
80 }

```

3.3 Closetpair

```

1 typedef pair<ll, ll> pii;
2 #define x first

```

```

3 #define y second
4 ll dd(const pii& a, const pii& b) {
5     ll dx = a.x - b.x, dy = a.y - b.y;
6     return dx * dx + dy * dy;
7 }
8 const ll inf = 1e18;
9 //在一點對陣列的[l, r]間找最近點對
10 ll dac(vector<pii>& p, int l, int r) {
11     if (l >= r) return inf;
12     int m = (l + r) / 2;
13     ll d = min(dac(p, l, m), dac(p, m + 1, r));
14     vector<pii> t;
15     for (int i = m; i >= l && p[m].x - p[i].x < d
        ; i--)
16         t.push_back(p[i]);
17     for (int i = m + 1; i <= r && p[i].x - p[m].x
        < d; i++)
18         t.push_back(p[i]);
19     sort(t.begin(), t.end(),
20         [](pii& a, pii& b) { return a.y < b.y;
        });
21     int n = t.size();
22     for (int i = 0; i < n - 1; i++)
23         for (int j = i + 1; j < n; j++)
24             // 這裡可以知道是哪兩點是最小點對
25             d = min(d, dd(t[i], t[j]));
26     return d;
27 }
28 // 給一堆點，求最近點對的距離「的平方」。
29 ll closest_pair(vector<pii>& pp) {
30     sort(pp.begin(), pp.end());
31     return dac(pp, 0, pp.size() - 1);
32 }

```

3.4 Gramh

```

1 //define pdd (double/int)
2 int cross(pdd a, pdd b){
3     return a.first*b.second - a.second*b.first;
4 }
5
6 pdd operator-(pdd a, pdd b){
7     return {a.first - b.first, a.second - b.
        second};
8 }
9
10 double operator*(pdd a, pdd b){
11     return a.first * b.second - a.second * b.
        first;
12 }
13
14 // ps是所有的點，要去重!!!
15 vector<pdd> convexHull(vector<pdd>& ps) {
16     sort(all(ps));
17     vector<pdd> hull;
18     if (ps.size() <= 2) {
19         return ps;
20     }
21     for (int i = 0; i < 2; i++) {
22         int s = hull.size();
23         for (pdd p : ps) {
24             while (hull.size() - s >= 2 && cross(
                hull.back() - hull[hull.size() -
                2], p - hull[hull.size() - 2]) < 1
                e-10) {
25                 hull.pop_back();
26             }
27             hull.pb(p); //push_back
28         }
29     }

```



```

29     hull.pop_back();
30     reverse(all(ps));
31 }
32 return hull;
33 }

```

4 Graph

4.1 LCA tarjan

```

1 // Tarjan (offline, O(n + m))
2 void dfs(int now, int p, int dep){
3     d[now] = dep; // d[i] = depth of i, be
4       careful about "root should set to 0/1"
5     for(int nxt:g[now]){
6         if(nxt==p) continue;
7         dfs(nxt, now, dep+1);
8         connect(now,nxt); // connect son "to" its
9           parent
10        vis[nxt] = 1;
11    }
12    // Deal with query
13    for(auto i:q[now]){
14        int nxt = i.first; // query has {now,nxt}
15        int id = i.second; // query_id
16        if(vis[nxt]){
17            qans[id] = find_root(nxt);
18        }
19    }
20 }

```

4.2 KuhnMunkres

```

1 const int N = 1005;
2 struct KM { // 0-base, maximum weight matching
3     ll w[N][N], hl[N], hr[N], slk[N];
4     int fl[N], fr[N], pre[N], qu[N], ql, qr, n;
5     bool vl[N], vr[N];
6     void init(int _n) {
7         n = _n;
8         for (int i = 0; i < n; ++i)
9             fill_n(w[i], n, -INF);
10    }
11    void add_edge(int a, int b, ll wei){
12        w[a][b] = wei;
13    }
14    bool Check(int x) {
15        if (vl[x] = 1, ~fl[x])
16            return vr[qu[qr++]] = fl[x] = 1;
17        while (~x) swap(x, fr[fl[x] = pre[x]]);
18        return 0;
19    }
20    void bfs(int s) {
21        fill_n(slk, n, INF), fill_n(vl, n, 0),
22        fill_n(vr, n, 0);
23        ql = qr = 0, qu[qr++] = s, vr[s] = 1;
24        for (ll d;;) {
25            while (ql < qr)
26                for (int x = 0, y = qu[ql++]; x <
27                  n; ++x)
28                    if (!vl[x] && slk[x] >= (d =
29                      hl[x] + hr[y] - w[x][y]))
30                        {
31                            if (pre[x] = y, d) slk[x]
32                                = d;
33                            else if (!Check(x))
34                                return;
35                        }
36        }
37    }
38 }

```

```

29     }
30     d = INF;
31     for (int x = 0; x < n; ++x)
32         if (!vl[x] && d > slk[x]) d = slk
33           [x];
34     for (int x = 0; x < n; ++x) {
35         if (vl[x]) hl[x] += d;
36         else slk[x] -= d;
37         if (vr[x]) hr[x] -= d;
38     }
39     for (int x = 0; x < n; ++x)
40         if (!vl[x] && !slk[x] && !Check(x)
41           ) return;
42 }
43 ll solve() {
44     fill_n(fl, n, -1), fill_n(fr, n, -1),
45     fill_n(hr, n, 0);
46     for (int i = 0; i < n; ++i)
47         hl[i] = *max_element(w[i], w[i] + n);
48     for (int i = 0; i < n; ++i) bfs(i);
49     ll res = 0;
50     for (int i = 0; i < n; ++i) res += w[i][
51       fl[i]];
52     return res;
53 }
54 } graph;
55 /*
56 Usage:
57 1. graph.init(n);
58 2. graph.add_edge(a, b, wei);
59 3. ll ans = graph.solve();
60 4. for (int i = 0; i < n; ++i) {
61     cout << "Matching " << i << " to " << graph.
62       fl[i] << " with weight " << graph.w[i][
63       graph.fl[i]] << endl;
64 }
65 if want to find minimum weight matching, just
66 negate the weight of all edges
67 and then the answer will be the negative of the
68 maximum weight matching
69 */

```

4.3 Hungarian

```

1 // Maximum Cardinality Bipartite Matching
2 // Worst case O(nm)
3
4 struct Graph{
5     static const int MAXN = 5003;
6     vector<int> G[MAXN];
7     int n, match[MAXN], vis[MAXN];
8
9     void init(int _n){
10        n = _n;
11        for (int i=0; i<n; i++) G[i].clear();
12    }
13
14    bool dfs(int u){
15        for (int v:G[u]){
16            if (vis[v]) continue;
17            vis[v]=true;
18            if (match[v]==-1 || dfs(match[v])){
19                match[v] = u;
20                match[u] = v;
21                return true;
22            }
23        }
24    }
25 }

```

```

24     return false;
25 }
26
27 int solve(){
28     int res = 0;
29     memset(match,-1,sizeof(match));
30     for (int i=0; i<n; i++){
31         if (match[i]==-1){
32             memset(vis,0,sizeof(vis));
33             if ( dfs(i) ) res++;
34         }
35     }
36     return res;
37 }
38 } graph;

```

4.4 MinCostMaxFlow

```

1  const int N = 2005;
2  struct MinCostMaxFlow { // 0-base
3      struct Edge {
4          ll from, to, cap, flow, cost, rev;
5      } *past[N];
6      vector<Edge> G[N];
7      int inq[N], n, s, t;
8      ll dis[N], up[N], pot[N];
9      bool BellmanFord() {
10         fill_n(dis, n, INF), fill_n(inq, n, 0);
11         queue<int> q;
12         auto relax = [&](int u, ll d, ll cap,
13             Edge *e) {
14             if (cap > 0 && dis[u] > d) {
15                 dis[u] = d, up[u] = cap, past[u]
16                     = e;
17                 if (!inq[u]) inq[u] = 1, q.push(u);
18             }
19         };
20         relax(s, 0, INF, 0);
21         while (!q.empty()) {
22             int u = q.front();
23             q.pop(), inq[u] = 0;
24             for (auto &e : G[u]) {
25                 ll d2 = dis[u] + e.cost + pot[u]
26                     - pot[e.to];
27                 relax(e.to, d2, min(up[u], e.cap
28                     - e.flow), &e);
29             }
30         }
31         return dis[t] != INF;
32     }
33     void solve(int _s, int _t, ll &flow, ll &cost
34         , bool neg = true) {
35         s = _s, t = _t, flow = 0, cost = 0;
36         if (neg) BellmanFord(), copy_n(dis, n,
37             pot);
38         for (; BellmanFord(); copy_n(dis, n, pot)
39             ) {
40             for (int i = 0; i < n; ++i) dis[i] +=
41                 pot[i] - pot[s];
42             flow += up[t], cost += up[t] * dis[t];
43             for (int i = t; past[i]; i = past[i]
44                 ->from) {
45                 auto &e = *past[i];
46                 e.flow += up[t], G[e.to][e.rev].
47                     flow -= up[t];
48             }
49         }
50     }

```

```

40 }
41 void init(int _n) {
42     n = _n, fill_n(pot, n, 0);
43     for (int i = 0; i < n; ++i) G[i].clear();
44 }
45 void add_edge(ll a, ll b, ll cap, ll cost) {
46     G[a].pb(Edge{a, b, cap, 0, cost, (ll)G[b]
47         .size()});
48     G[b].pb(Edge{b, a, 0, 0, -cost, (ll)G[a].
49         size() - 1});
50 }
51 };
52 /*
53 Usage:
54 1. mcmf.init(n); // Initialize, n = number of
55     nodes (0-based indexing)
56 2. mcmf.add_edge(u, v, cap, cost); // Add edge
57     with capacity `cap` and per-unit cost `cost`
58 3. mcmf.solve(s, t, flow, cost, neg); // Run
59     Min-Cost Max-Flow
60     - s = source
61     - t = sink
62     - flow = maximum flow (output)
63     - cost = corresponding minimum cost (output)
64     - neg = whether negative-cost edges may exist;
65         default = true.
66 Example:
67 MinCostMaxFlow mcmf;
68 mcmf.init(4);
69 mcmf.add_edge(0,1,2,1);
70 mcmf.add_edge(0,2,1,2);
71 mcmf.add_edge(1,2,1,0);
72 mcmf.add_edge(1,3,1,3);
73 mcmf.add_edge(2,3,2,1);
74 ll flow, cost;
75 mcmf.solve(0, 3, flow, cost);
76 // flow = 3, cost = 8
77 */

```

4.5 Bridge

```

1 //for undirected graph, find bridge
2 const int N = 1e6+5;
3
4 vector<pair<int,int> > edge(N); //{u,v} ->
5     remember to input
6 vector<pair<int,int> > g[N]; //{nxt,edge_id}
7 vector<int> bridge(N);
8 int dfn[N],vis[N],low[N],id;
9 void tarjan(int now,int p){
10     dfn[now]=id++;
11     vis[now]=1;
12     low[now]=dfn[now];
13     for(auto [nxt,id]:g[now]){
14         if(nxt==p) continue;
15         if(vis[nxt]){
16             low[now]=min(low[now],dfn[nxt]); //
17                 back edge!
18         }
19         else{
20             tarjan(nxt,now);
21             low[now]=min(low[now],low[nxt]);
22             if(low[nxt]>dfn[now]){
23                 bridge[id]=1;
24             }
25         }
26     }
27 }

```

```

26 signed main(){
27     // construct
28     for(int i=0;i<n;i++){
29         dfn[i]=1e9;//reset
30         low[i]=1e9;
31         vis[i]=0;
32     }
33     id=0;
34     for(int i=0;i<n;i++){
35         if(!vis[i]){
36             tarjan(i,-1);
37         }
38     }
39     // use
40 }

```

4.6 LCA fd

```

1 // online O(nlogn + mlogn)
2 const int N = 300005;
3 int d[N],f[N][20]; // f[i][j] = i's 2^j father, d[i]
4 // i] = depth of i
5 vector<int> g[N]; // graph
6 void dfs(int now,int p,int dep){
7     d[now] = dep+1;
8     for(int nxt:g[now]){
9         if(nxt==p) continue;
10        f[nxt][0] = now;
11        dfs(nxt,now,dep+1);
12    }
13    return;
14 }
15
16 int lca(int x, int y){
17     if(d[x]<d[y]) swap(x,y);
18     int k = d[x]-d[y];
19     for(int i=0;i<20;i++){
20         if(k&1) x = f[x][i];
21         k>>=1;
22     } // jump to the same depth/height
23     if(x==y) return x;
24     for(int i=19;i>=0;i--){
25         if(f[x][i]!=f[y][i]){
26             x = f[x][i];
27             y = f[y][i];
28         }
29     } // find the first different -> higher is LCA
30     return f[x][0];
31 }
32 void sol(){
33     int n,m;
34     cin>>n>>m;
35     for(int i=1;i<n;i++){
36         int u,v;
37         cin>>u>>v;
38         g[u].pb(v);
39         g[v].pb(u);
40     }
41     dfs(1,0,0); // arbitrarily choose a root,
42     // here choose 1 as root
43     for(int j=1;j<20;j++){
44         for(int i=1;i<=n;i++){
45             f[i][j] = f[f[i][j-1]][j-1];
46         } // get all f
47     }
48     // --- use lca(u,v) to get ---
49 }

```

4.7 Dinic

```

1 (a) Bounded Maxflow Construction:
2 1. add two node ss, tt
3 2. add_edge(ss, tt, INF)
4 3. for each edge u -> v with capacity [l, r]:
5     add_edge(u, tt, l)
6     add_edge(ss, v, l)
7     add_edge(u, v, r-l)
8 4. see (b), check if it is possible.
9 5. answer is maxflow(ss, tt) + maxflow(s, t)
10 -----
11 (b) Bounded Possible Flow:
12 1. same construction method as (a)
13 2. run maxflow(ss, tt)
14 3. for every edge connected with ss or tt:
15     rule: check if their rest flow is exactly
16         0
17 4. answer is possible if every edge do satisfy
18     the rule;
19 5. otherwise, it is NOT possible.
20 -----
21 (c) Bounded Minimum Flow:
22 1. same construction method as (a)
23 2. answer is maxflow(ss, tt)
24 -----
25 (d) Bounded Minimum Cost Flow:
26 * the concept is somewhat like bounded possible
27     flow.
28 1. same construction method as (a)
29 2. answer is maxflow(ss, tt) + (Σ l * cost for
30     every edge)
31 -----
32 (e) Minimum Cut:
33 1. run maxflow(s, t)
34 2. run cut(s)
35 3. ss[i] = 1: node i is at the same side with s.
36 -----
37 const long long INF = 1LL<<60;
38 struct Dinic { //O(VVE), with minimum cut
39     static const int MAXN = 5003;
40     struct Edge{
41         int u, v;
42         long long cap, rest;
43     };
44
45     int n, m, s, t, d[MAXN], cur[MAXN];
46     vector<Edge> edges;
47     vector<int> G[MAXN];
48
49     void init(){
50         edges.clear();
51         for ( int i = 0 ; i < n ; i++ ) G[i].
52             clear();
53         n = 0;
54     }
55
56     // min cut start
57     bool side[MAXN];
58     void cut(int u) {
59         side[u] = 1;
60         for ( int i : G[u] ) {
61             if ( !side[ edges[i].v ] && edges[i].
62                 rest ) cut(edges[i].v);
63         }
64     }
65 }

```

```

58     }
59 }
60 // min cut end
61
62 int add_node(){
63     return n++;
64 }
65
66 void add_edge(int u, int v, long long cap){
67     edges.push_back( {u, v, cap, cap} );
68     edges.push_back( {v, u, 0, 0LL} );
69     m = edges.size();
70     G[u].push_back(m-2);
71     G[v].push_back(m-1);
72 }
73
74 bool bfs(){
75     fill(d,d+n,-1);
76     queue<int> que;
77     que.push(s); d[s]=0;
78     while (!que.empty()){
79         int u = que.front(); que.pop();
80         for (int ei : G[u]){
81             Edge &e = edges[ei];
82             if (d[e.v] < 0 && e.rest > 0){
83                 d[e.v] = d[u] + 1;
84                 que.push(e.v);
85             }
86         }
87     }
88     return d[t] >= 0;
89 }
90
91 long long dfs(int u, long long a){
92     if ( u == t || a == 0 ) return a;
93     long long flow = 0, f;
94     for ( int &i=cur[u]; i < (int)G[u].size(); i++ ) {
95         Edge &e = edges[ G[u][i] ];
96         if ( d[u] + 1 != d[e.v] ) continue;
97         f = dfs(e.v, min(a, e.rest) );
98         if ( f > 0 ) {
99             e.rest -= f;
100             edges[ G[u][i]^1 ].rest += f;
101             flow += f;
102             a -= f;
103             if ( a == 0 ) break;
104         }
105     }
106     return flow;
107 }
108
109 long long maxflow(int _s, int _t){
110     s = _s, t = _t;
111     long long flow = 0, mf;
112     while ( bfs() ){
113         fill(cur,cur+n,0);
114         while ( (mf = dfs(s, INF)) ) flow += mf;
115     }
116     return flow;
117 }
118 } dinic;

```

4.8 2sat tarjan

```

1 const int N = 2005;
2 int low[N],dfn[N],color[N],ins[N]; //要開兩倍大
3 // color[x] 是 x 所在的 scc 的topo逆序。

```

```

4 vector<int> g[N];
5 int dfsClock,sccCnt;
6 stack<int> stk;
7 void tarjan(int u) {
8     low[u] = dfn[u] = ++dfsClock;
9     stk.push(u); ins[u] = true;
10    for (const auto &v : g[u]) {
11        if (!dfn[v]) tarjan(v), low[u] = std::min(
12            low[u], low[v]);
13        else if (ins[v]) low[u] = std::min(low[u],
14            dfn[v]);
15    }
16    if (low[u] == dfn[u]) {
17        ++sccCnt;
18        do {
19            color[u] = sccCnt;
20            u = stk.top(); stk.pop(); ins[u] =
21                false;
22        } while (low[u] != dfn[u]);
23    }
24 }
25
26 signed main(){
27     g[i].pb(j); // i->j
28
29     for (int i = 1; i <= (n << 1); ++i) if (!dfn[
30         i]) tarjan(i); // run tarjan, 注意0~2n-1
31         or 1~2n
32
33     for(int i=1;i<=n;i++){
34         if(color[i] == color[i+n]){
35             cout<<"NO"<<endl;
36             return;
37         }
38     }
39     cout<<"YES"<<endl;
40     // 找環 注意建邊方法是(i,i+1) or (i,i+n)
41
42     for(int i=1;i<=n;i++){
43         if(color[i] < color[i+n]){
44             cout<<1<<" ";
45         }
46         else cout<<0<<" ";
47     }
48     // 構造解 (注意是0~n-1還是1~n)
49 }

```

4.9 Hopcroft

```

1 // O(sqrt(V)E)
2 struct Graph{
3     static const int MAXN = 4005;
4     vector<int> G[MAXN];
5     int N1, N2, match[MAXN], dist[MAXN];
6
7     void init(int _n1, int _n2){
8         N1 = _n1;
9         N2 = _n2;
10        for (int i = 0; i < N1 + N2; i++) G[i].
11            clear();
12    }
13
14    void addEdge(int u, int v){
15        G[u].pb(v + N1);
16    }
17
18    bool bfs(){
19        queue<int> Q;
20        for (int u = 0; u < N1; u++){

```

```

20         if (match[u] == -1){
21             dist[u] = 0;
22             Q.push(u);
23         } else {
24             dist[u] = -1;
25         }
26     }
27     bool found = false;
28     while (!Q.empty()){
29         int u = Q.front(); Q.pop();
30         for (int v : G[u]){
31             int m = match[v];
32             if (m != -1 && dist[m] != -1)
33                 continue;
34             if (m != -1){
35                 dist[m] = dist[u] + 1;
36                 Q.push(m);
37             } else {
38                 found = true;
39             }
40         }
41     }
42     return found;
43 }
44 bool dfs(int u){
45     for (int v : G[u]){
46         int m = match[v];
47         if (m == -1 || (dist[m] == dist[u] +
48             1 && dfs(m))){
49             match[u] = v;
50             match[v] = u;
51             return true;
52         }
53     }
54     dist[u] = -1;
55     return false;
56 }
57 int solve(){
58     int res = 0;
59     memset(match, -1, sizeof(match));
60     while (bfs()){
61         for (int u = 0; u < N1; u++){
62             if (match[u] == -1){
63                 if (dfs(u)) res++;
64             }
65         }
66     }
67     return res;
68 }
69 } graph;
70
71 /*
72 graph.init(n1, n2) : 2 parts' node numbers
73 graph.addEdge(i, j) : edge i->j (不須bias)
74 graph.solve() : return match numbers
75 graph.match[i] : match AFTER bias!
76 */

```

4.10 2e cc

```

1 // i.e. bridge tree
2 // Remember to reset vis[]
3 map<int,int> compId;
4 vector<int> g2[N];
5 void dfs(int now,int p,int iid){
6     vis[now]=1;
7     compId[now]=iid;

```

```

8     for(auto [nxt,id]:g[now]){
9         if(bridge[id]) continue;
10        if(nxt==p) continue;
11        if(!vis[nxt]) dfs(nxt,now,iid);
12    }
13 }
14 //Then, in main()
15 int iid=0;
16 for(int i=0;i<n;i++) vis[i]=0;
17 for(int i=0;i<n;i++){
18     if(!vis[i]){
19         dfs(i,-1,iid);
20         iid+=1;
21     }
22 }
23 for(int i=0;i<m;i++){
24     if(bridge[i]){
25         auto [u,v] = edge[i];
26         g2[compId[u]].pb(compId[v]);
27         g2[compId[v]].pb(compId[u]);
28     }
29 }

```

5 Math

5.1 FFT

```

1 const int N = 1e7+10;
2 const double Pi = acos(-1.0);
3 struct Complex{
4     double x,y;
5     Complex ( double xx=0, double yy=0){
6         x=xx;
7         y=yy;
8     }
9 };
10 Complex a[N], b[N];
11 Complex operator + (Complex a, Complex b) {
12     return Complex(a.x + b.x , a.y + b.y);
13 }
14 Complex operator - (Complex a, Complex b) {
15     return Complex(a.x - b.x , a.y - b.y);
16 }
17 Complex operator * (Complex a, Complex b) {
18     return Complex(a.x * b.x - a.y * b.y , a.x * b
19         .y + a.y * b.x);
20 }
21 int limit=1, h=0, rev[N];
22 void fft(Complex *A, int flag){
23     for(int i=0; i<limit; i++){
24         if(i<rev[i]){
25             swap(A[i], A[rev[i]]);
26         }
27     }
28     for(int len=1; len<limit; len<=<=1){
29         // Len = 待合併區間的一半
30         Complex Wn(cos(Pi/len), flag*sin(Pi/len))
31             ;
32         for(int R=len<<1, j=0; j<limit; j+=R){
33             Complex w(1, 0);
34             for (int k = 0; k < len; k++, w = w *
35                 Wn) {
36                 Complex x = A[j + k], y = w * A[j
37                     + len + k];
38                 A[j + k] = x + y;
39                 A[j + len + k] = x - y;
40             }
41         }
42     }
43 }

```

```

35 }
36
37 int main(){
38     int n,m; // n,m次方
39     cin>>n>>m;
40     for(int i=0; i<=n; i++) cin>>a[i].x;
41     for(int i=0; i<=m; i++) cin>>b[i].x;
42     while(limit<=n+m){
43         limit=limit<<1;
44         h++;
45     }
46     for(int i=0; i<limit; i++){
47         rev[i] = (rev[i>>1]>>1) | ((i&1)<<(h-1));
48         // bit reverse
49     }
50     fft(a, 1);
51     fft(b, 1); //FFT
52     for(int i=0; i<=limit; i++) a[i]=a[i]*b[i];
53     fft(a, -1); //IFFT
54     for(int i=0; i<=n+m; i++) cout<<(int)(a[i].x
55         / limit + 0.5)<<" ";
56 }

```

5.2 RabinMiller

```

1 #include <bits/stdc++.h>
2 #define int long long
3 using namespace std;
4
5 int QuickPow(int base, int exponent, int mod){
6     if(exponent == 0)
7         return 1;
8     if(exponent == 1)
9         return base;
10    if(exponent % 2)
11        return QuickPow(base, exponent - 1, mod)
12            * base % mod;
13    int tmp = QuickPow(base, exponent / 2, mod);
14    return tmp * tmp % mod;
15 }
16
17 bool RabinMiller(int d, int n){
18     int a = 2 + rand() % (n - 2);
19     if(QuickPow(a, n - 1, n) != 1)
20         return false;
21     int cur = QuickPow(a, d, n);
22     int nx;
23     while(d != n - 1){
24         nx = (cur * cur) % n;
25         d *= 2;
26         if(cur != 1 && cur != n - 1){
27             if(nx == 1)
28                 return false;
29         }
30         cur = nx;
31     }
32     return true;
33 }
34
35 bool isPrime(int n, int k){
36     if(n <= 1)
37         return false;
38     if(n <= 3)
39         return true;
40     if(n == 4)
41         return false;
42     if((n - 1) % 6 != 0 && (n + 1) % 6 != 0)
43         return false;
44     int d = n - 1;

```

```

44     int r = 0;
45     while(d % 2 == 0){
46         d /= 2;
47         r ++;
48     }
49     for(int i=0; i<k; i++){
50         if(!RabinMiller(d, n))
51             return false;
52     }
53     return true;
54 }
55
56 signed main(){
57     int n;
58     while(cin >> n){
59         if(isPrime(n, 5)) // 預設 k = 5
60             cout << "質數" << endl;
61         else
62             cout << "非質數" << endl;
63     }
64 }

```

5.3 LinearSieve

```

1 int LeastPrimeDivisor[maxn];
2 vector<int> pr;
3
4 void LinearSieve(){
5     for(int i = 2; i < maxn; i++){
6         if(!LeastPrimeDivisor[i]) pr.push_back(i),
7             LeastPrimeDivisor[i] = i;
8         for(int p : pr){
9             if(i * p >= maxn) break;
10            LeastPrimeDivisor[i * p] = p;
11            if(i % p == 0) break;
12        }
13    }
14 }

```

5.4 Rho

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 __int128 n;
5 __int128 gcd(__int128 a, __int128 b) {
6     if (b == 0) return a;
7     return gcd(b, a % b);
8 }
9
10 __int128 read() {
11     __int128 x = 0, f = 1;
12     char ch = getchar();
13     while (ch < '0' || ch > '9') {
14         if (ch == '-') f = -1;
15         ch = getchar();
16     }
17     while (ch >= '0' && ch <= '9') {
18         x = x * 10 + ch - '0';
19         ch = getchar();
20     }
21     return x * f;
22 }
23
24 void print(__int128 x) {
25     if (x < 0) {
26         putchar('-');
27         x = -x;

```

```

28     }
29     if (x > 9) print(x / 10);
30     putchar(x % 10 + '0');
31 }
32 bool cmp(__int128 x, __int128 y) { return x > y;
33     }
34 __int128 quick_pow(__int128 x, __int128 p,
35     __int128 mod) {
36     __int128 ans = 1;
37     while (p) {
38         if (p & 1) ans = (__int128)ans * x % mod;
39         x = (__int128)x * x % mod;
40         p >>= 1;
41     }
42     return ans;
43 }
44 bool Miller_Rabin(__int128 p) {
45     if (p < 2) return 0;
46     if (p == 2) return 1;
47     if (p == 3) return 1;
48     __int128 d = p - 1, r = 0;
49     while (!(d & 1)) ++r, d >>= 1;
50     for (int k = 0; k < 10; ++k) {
51         __int128 a = (__int128)rand() % (p - 2) + 2;
52         __int128 x = quick_pow(a, d, p);
53         if (x == 1 || x == p - 1) continue;
54         for (int i = 0; i < r - 1; ++i) {
55             x = (__int128)x * x % p;
56             if (x == p - 1) break;
57         }
58         if (x != p - 1) return 0;
59     }
60     return 1;
61 }
62
63 __int128 Pollard_Rho(__int128 x) {
64     __int128 s = 0, t = 0;
65     __int128 c = (__int128)rand() % (x - 1) + 1;
66     __int128 step = 0, goal = 1;
67     __int128 val = 1;
68     for (goal = 1; goal * 2 <= x; goal *= 2, s = t, val = 1) {
69         for (step = 1; step <= goal; ++step) {
70             t = ((__int128)t * t + c) % x;
71             __int128 ts;
72             if (t >= s) ts = t - s;
73             else ts = s - t;
74             val = (__int128)val * ts % x;
75             if ((step % 127) == 0) {
76                 __int128 d = gcd(val, x);
77                 if (d > 1) return d;
78             }
79         }
80         __int128 d = gcd(val, x);
81         if (d > 1) return d;
82     }
83 }
84
85 set<__int128> st;
86 void fac(__int128 x){
87     if(x<2) return;
88     if(Miller_Rabin(x)){
89         st.insert(x);
90         return;
91     }
92     __int128 p = x;
93     while(p>=x) p = Pollard_Rho(x);
94     while((x%p) == 0) x /= p;
95     fac(p); fac(x);

```

```

96 }
97
98 vector<__int128> v; // prime factors
99 set<__int128> allf; // factors
100 void dfs(__int128 res, __int128 now){
101     auto it = allf.find(now);
102     if(it != allf.end()) return;
103     allf.insert(now);
104     if(res < 2) return;
105     for(__int128 i:v){
106         if(res % i == 0){
107             dfs(res / i, now * i);
108         }
109     }
110 }
111
112 int main() {
113     srand((unsigned)time(NULL));
114     n = read();
115     __int128 N = (__int128)(n) * n;
116     fac(n); // find all n's prime factor
117     for(__int128 i:st) v.push_back(i);
118     dfs(N, 1);
119     // allf = all N's factors
120     for(auto i:allf){
121         print(i); cout<<endl;
122     }
123 }

```

5.5 NTT

```

1 const int N = 1e7+10;
2 const int P = 998244353, G = 3, Gi = 332748118; //
   primitive root = 3, Gi = mod inverse of 3
3
4 int fastpow(int x,int p){
5     int sum = 1;
6     while(p){
7         if(p&1) sum = sum*x%P;
8         x = x*x%P;
9         p = p>>1;
10    }
11    return sum;
12 }
13
14 int a[N], b[N], limit=1, h=0, rev[N];
15 inline void NTT(int *A, int flag) {
16     for(int i = 0; i < limit; i++)
17         if(i < rev[i]) swap(A[i], A[rev[i]]);
18     for(int len = 1; len < limit; len <= 1){
19         int Wn = fastpow( flag == 1 ? G : Gi , (P -
20             1) / (len <= 1));
21         for(int j = 0; j < limit; j += (len <= 1)){
22             int w = 1;
23             for(int k = 0; k < len; k++) {
24                 int x = A[j + k], y = w * A[j + k
25                     + len] % P;
26                 A[j + k] = (x + y) % P,
27                 A[j + k + len] = (x - y + P) % P;
28                 w = (w * Wn) % P;
29             }
30         }
31     }
32 }
33
34 signed main(){
35     int n,m;
36     cin>>n>>m; // n,m次方
37     for(int i=0; i<=n; i++){

```



```

36     cin>>a[i];
37     a[i]=(a[i] + P) % P;
38 }
39 for(int i=0; i<=m; i++){
40     cin>>b[i];
41     b[i]=(b[i] + P) % P;
42 }
43 while(limit<=n+m){
44     limit=limit<<1;
45     h++;
46 }
47 for(int i=0; i<limit; i++){
48     rev[i] = (rev[i>>1]>>1) | ((i&1)<<(h-1));
49 }
50 NTT(a, 1);
51 NTT(b, 1);
52 for(int i=0; i<=limit; i++) a[i]=a[i]*b[i]%P;
53 NTT(a, -1);
54
55 int inv = fastpow(limit, P - 2);
56 for(int i=0; i<=n+m; i++){
57     cout<<(a[i]*inv)%P << " ";
58 }
59 }

```

6 Misc

6.1 custom hash

```

1 #include<bits/stdc++.h>
2 #define ll long long int
3 using namespace std;
4
5 struct custom_hash {
6     static uint64_t splitmix64(uint64_t x) {
7         x += 0x9e3779b97f4a7c15;
8         x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
9         x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
10        return x ^ (x >> 31);
11    }
12
13    size_t operator()(ll x) const {
14        static const uint64_t FIXED_RANDOM =
15            chrono::steady_clock::now().
16            time_since_epoch().count();
17        return splitmix64(x + FIXED_RANDOM);
18    }
19 };
20 // unordered_map<ll, int, custom_hash> mp;

```

6.2 Rand

```

1 mt19937_64 rnd(random_device{}());
2 uniform_int_distribution<int> dist(i, j);
3 /*
4 dist(rnd) -> 取 i~j範圍內的整數
5 */

```

6.3 SG

```

1 int f[100]; // state, reset to -1
2
3 int sg(int x) {
4     if (f[x] != -1) return f[x];
5     unordered_set<int> S;

```

```

6     if (x >= 1) S.insert(sg(x - 1)); // all sub-
7         states (suppose x can be x-1, x-2, 0)
8     if (x >= 2) S.insert(sg(x - 2));
9     if (x >= 3) S.insert(sg(0));
10    for (int i = 0; i++) {
11        if (!S.count(i)) return f[x] = i;
12    }

```

7 Strings

7.1 SuffixArray

```

1 struct SuffixArray{
2     void countSort(vector<int> &pos, vector<int>
3         &rank){
4         int n = pos.size();
5         vector<vector<int>> cnt(n);
6         for(auto i : pos) cnt[rank[i]].push_back
7             (i);
8         for(int i=0,idx=0;i<n;i++){
9             for(auto x : cnt[i])
10                pos[idx++] = x;
11        }
12    }
13    void getSuffix(string s, vector<int> &pos,
14        vector<int> &lcp){
15        s += '$';
16        int n = s.size();
17        vector<int> rank(n);
18        iota(pos.begin(), pos.end(), 0);
19        sort(pos.begin(), pos.end(), [&](int a,
20            int b){ return s[a] < s[b]; });
21        // k = 0
22        for(int i=0;i<n;i++){
23            if(i == 0)
24                rank[pos[i]] = 0;
25            else
26                rank[pos[i]] = rank[pos[i-1]] + (
27                    s[pos[i]] != s[pos[i-1]]);
28        }
29        // k > 0
30        vector<int> newRank(n);
31        for(int k=0;(1<<k)<=n;k++){
32            for(int i=0;i<n;i++){
33                pos[i] = (pos[i] - (1 << k) % n +
34                    n) % n;
35                countSort(pos, rank);
36                newRank[pos[0]] = 0;
37                for(int i=1;i<n;i++){
38                    pair<int,int> prev = {rank[pos[i-1]], rank
39                        [(pos[i-1]+(1<<k))%n]};
40                    pair<int,int> now = {rank[pos[i]], rank
41                        [(pos[i]+(1<<k))%n]};
42                    newRank[pos[i]] = newRank[pos[i-1]] + (prev!=now);
43                }
44                rank = newRank;
45            }
46        }
47        // build lcp
48        int k = 0;
49        for(int i = 0; i < n; i++){
50            int pi = rank[i];
51            int j = pos[pi-1];
52            while(i + k < n && j + k < n && s[i+k]
53                == s[j+k]) k++;
54            lcp[pi] = k;

```

```

45         k = max(0, k-1);
46     }
47 }
48 } sa;
49
50 /*
51 vector<int> pos, lcp;
52 sa.getSuffix(s, pos, lcp);
53 */

```

7.2 KMP

```

1 vector<int> Failure(const string &s){
2     // initialize the failure function
3     vector<int> f(s.size(), 0);
4     int k = 0;
5     for (int i = 1; i < (int)s.size(); ++i) {
6         while (k > 0 && s[i] != s[k]) k = f[k -
7             1];
8         if (s[i] == s[k]) ++k;
9         f[i] = k;
10    }
11    return f;
12 }
13
14 vector<int> Matching(const string &s, const
15     string &t){
16     // return 0-indexed occurrence of t in s
17     // O(N + M)
18     vector<int> f = Failure(t);
19     int l = 0; // for the index of s
20     int r = 0; // for the index of t(pattern)
21     vector<int> res ;
22     while(l < s.size()){
23         if(s[l] == t[r]){
24             l++;
25             r++;
26             if(r == t.size()){
27                 res.push_back(l - t.size());
28                 r = f[r-1];
29             }
30         }
31         else{
32             if(r == 0){
33                 l++;
34             }
35             else{
36                 r = f[r-1];
37             }
38         }
39     }
40     return res;
41 }

```

7.3 Rolling hash

```

1 const int N = 2000; // string size
2 int X = 1000000007, P = 4000000007;
3 // other primes : 8298176713 5119240589
4 // 3735751997 3218996237...
5
6 int s[N], p[N]; // if hash兩次 -> 開兩個s,p
7
8 void Hash(string& str){
9     s[0] = str[0];
10    p[0] = 1;
11    for(int i=1; i<str.size(); i++){
12        s[i] = (s[i-1]*X + str[i])%P;
13        p[i] = (p[i-1]*X) % P;
14    }
15 }

```

```

12 }
13 return;
14 }
15 int hash_i(int a,int b){
16     if(!a) return s[b];
17     int tmp = s[b] - s[a-1]*p[b-a+1]; //
18         calculate H[a:b]
19     tmp=tmp%P;
20     if(tmp<0) return tmp+P;
21     return tmp;
22 }
23
24 signed main(){
25     // Hash to generate s and p
26     // hash_i to calculate s[a:b]
27     string s;
28     cin>>s;
29     Hash(s);
30     cout<<hash_i(0,2)<<" "<<hash_i(3,5);
31 }

```

ACM ICPC Team - NYCU OverFlowers

Contents

1 Basic	1		
1.1 Default	1		
2 DataStructure	1		
2.1 Mo	1		
2.2 Lazy Seg	1		
2.3 BIT query	2		
2.4 PBDS	2		
2.5 Treap	2		
2.6 DSU set	3		
2.7 Per Seg	4		
2.8 DSU f	5		
2.9 SparseTable	5		
2.10 01trie	5		
2.11 Dy Seg	6		
3 Geometric	7		
3.1 Rectangle Union Area	7		
3.2 TheLeastCoverCircle	7		
3.3 Closetpair	8		
		3.4 Gramh	8
4 Graph	9		
4.1 LCA tarjan	9		
4.2 KuhnMunkres	9		
4.3 Hungarian	9		
4.4 MinCostMaxFlow	10		
4.5 Bridge	10		
4.6 LCA fd	11		
4.7 Dinic	11		
4.8 2sat tarjan	12		
4.9 Hopcroft	12		
4.10 2e cc	13		
5 Math	13		
5.1 FFT	13		
5.2 RabinMiller	14		
5.3 LinearSieve	14		
5.4 Rho	14		
5.5 NTT	15		
6 Misc	16		
6.1 custom hash	16		
6.2 Rand	16		
6.3 SG	16		
7 Strings	16		
7.1 SuffixArray	16		
7.2 KMP	17		
7.3 Rolling hash	17		